QA AUTOMATION & TEST ENGINEERING PORTFOLIO

Enterprise Portal Test Automation Framework

A resilient, layered Keyword-Driven E2E test automation architecture built with Robot Framework, Selenium WebDriver, and REST API integration for large-scale enterprise workflows.

AUTHOR & ENGINEER

Kharis Ediwardana

CORE TECHNOLOGIES

Robot Framework, Selenium, Python

DESIGN PATTERN

Layered Keyword-Driven / Page Object

CI/CD & EXECUTION

GitHub Actions, Headless Chrome

1. Executive Summary & Project Overview

Modern enterprise systems often bridge complex public-facing citizen self-service portals with rigorous back-office case management platforms. Ensuring reliability across high-stakes digital submissions (such as dispute appeals, legal notices, and multi-format document verification) requires an automated regression harness that is resilient, maintainable, and free from intermittent test flakiness.

75%	100+	20+	<1%
REGRESSION CYCLE REDUCTION	AUTOMATED TEST CASES	FILE UPLOAD FORMATS TESTED	INTERMITTENT TEST FLAKINESS

Business Problem

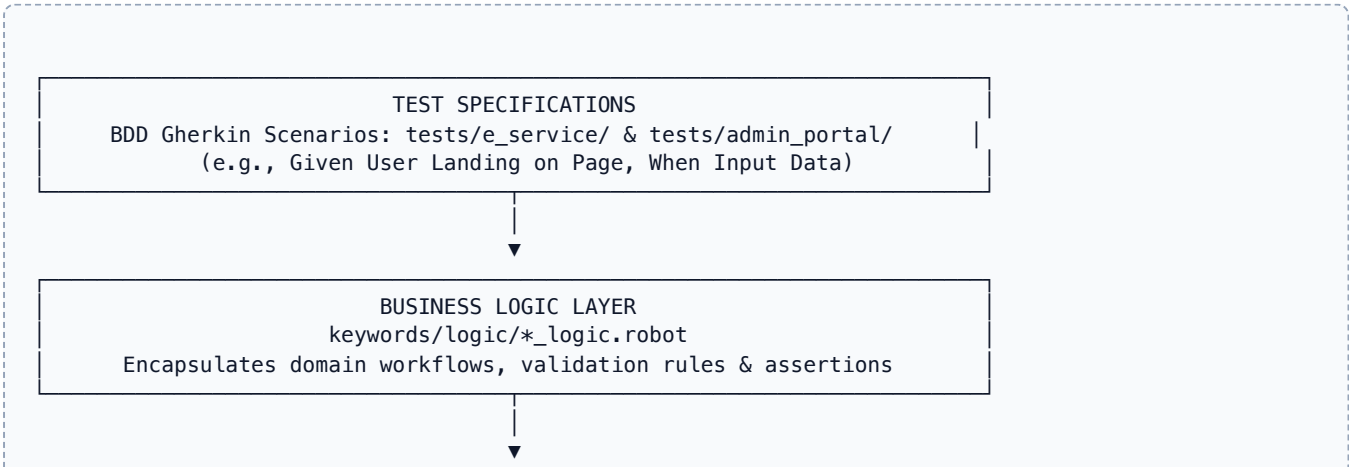
- **Extensive Manual Testing Burden:** Regression verification for multi-step portal submissions required over 3 days of manual testing prior to each release cycle.
- **Asynchronous UI Complexity:** Heavy modal dialogues, role-based conditional inputs, and file upload pipelines frequently triggered timing issues in naive automation scripts.
- **Multi-Environment Drift:** Deployments moved across multiple staging environments (SIT and Staging), requiring zero-downtime test configuration shifts without code modifications.

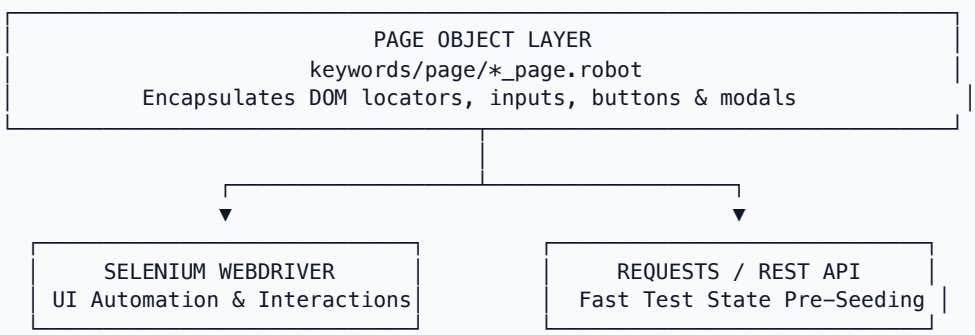
Automated Solution

An end-to-end automated testing suite was engineered leveraging **Robot Framework**, **Selenium WebDriver**, **RequestsLibrary**, and **FakerLibrary**. By abstracting element locators into a Page Object layer and orchestrating business user journeys in a reusable Logic layer, the framework enables fast, human-readable test authoring while maintaining strict architectural decoupling.

2. Framework Architecture & Design Pattern

The framework strictly adheres to a **Layered Keyword-Driven Architecture** combined with the **Page Object Model (POM)**. This guarantees that UI updates (such as CSS selector modifications) are localized to a single file without breaking upstream business test cases.





📁 keywords/page/ (Page Objects)

Houses concrete web element locators (ID, XPath, CSS) and primitive browser actions. Isolates HTML structure changes from test scripts.

⚙️ keywords/logic/ (Business Logic)

Orchestrates reusable business steps (e.g., logging in as a specific role, submitting multi-page appeals, checking audit trails).

🧪 tests/ (Executable Scenarios)

Declarative test cases written using business terminology and tagging (`Smoke`, `Regression`, `SprintTag`).

🛠️ helper.robot (Cross-Cutting Utilities)

Viewport scrolling scripts, dynamic mock data generators (FakerLibrary), and dynamic wait wrappers.

3. Core Engineering Highlights

1. Resilient Dynamic Wait Strategy (Eliminating Flakiness)

Arbitrary, hardcoded `Sleep` statements are the primary culprit behind test flakiness and bloated CI/CD runtimes. This framework implements a 4-tier conditional dynamic wait hierarchy:

Wait Tier	Threshold	Purpose & Application
TURBO_TIMEOUT	1s	Micro-waits for inline CSS animations, DOM transitions, and dropdown expansions.
SHORT_TIMEOUT	3s	Standard input element visibility, button enabling, and toast notification checks.
MEDIUM_TIMEOUT	5s	Modal dialogue appearances, async data table row populations, and tab switches.
LONG_TIMEOUT	10s	Heavy payload ingestion, server-side PDF generation, and batch export downloads.

2. Multi-Format File Attachment Verification Matrix

Enterprise citizen portals must strictly regulate user attachments. The test suite includes an automated matrix in `Media/` to test boundary conditions across 20+ file formats:

- **Valid Formats:** `.pdf`, `.docx`, `.xlsx`, `.csv`, `.png`, `.jpg`, `.zip`.
- **Negative & Boundary Testing:** Unsupported extensions (`.zippp`, `.exe`), empty payloads (`Empty.csv`), corrupted records (`invalidNotice.csv`), and oversized media files (`>10MB`).
- **Automated Batch Processing:** Testing automated CSV ingestion with dynamic parsing of multiple notice records in a single batch pass.

3. Hybrid UI & REST API Automated State Provisioning

Testing deep UI workflows often requires an entity to exist in a specific lifecycle state (e.g., an unpaid notice or escalated dispute). Rather than performing repetitive UI steps, the framework uses **RequestsLibrary** (`tests/api/api_test.robot`) to seed database records in milliseconds:

- `POST /sessions`: Immediately provisions an active parking/case session.
- `PATCH /records/{id}`: Modifies timestamp and status flags to trigger automated appeal eligibility rules.
- `GET /sync-data/pull-case`: Triggers backend microservice sync before initiating UI verification.

4. Multi-Environment Isolation & Zero Hardcoding

Configuration is externalized into dedicated files (`config/environment_sit.robot` and `config/environment_staging.robot`). Test suites dynamically load endpoints and credentials via CLI variables without altering source code.

4. Code Architecture & Implementation Walkthrough

Declarative BDD Test Case Specification

```
*** Settings ***
Documentation    Citizen Portal Appeal Submission & Dispute Management
Resource         ../../resources.robot
Suite Setup      Open Browser And Go To Public Portal
Test Setup       Navigate To Public Portal
Test Teardown    Capture Page If Test Failed
Suite Teardown   Close All Browsers
Force Tags       AppealPortal

*** Test Cases ***
PLUS_12.1_001 User Creates New Dispute Case With Valid Particulars
    [Tags]    Smoke    Regression
    Given User Landing on Public Appeal Page
    When User Accepts Terms and Declaration
    And User Inputs Appeal Notice Particulars
    And User Uploads Supporting Evidence Document
    Then User Able Submit Create New Case Data
    And System Displays Successful Acknowledgment Number
```

Business Logic Encapsulation (`keywords/logic/`)

```
*** Keywords ***
User Inputs Appeal Notice Particulars
    [Arguments]
    Wait And Input Text              timeout=
    Click Element
    Wait Until Element Is Visible        timeout=

User Uploads Supporting Evidence Document
    [Arguments]
    # Resilient file upload without relying on native OS dialogue
    Scroll Element Into View
    Choose File
    Wait Until Page Contains Element    xpath=//*[contains(text(),"Upload Successful")]
```

Automated Teardown & Failure Diagnostics (`tests/TestTeardown.robot`)

```
*** Keywords ***
Capture Page If Test Failed
    [Documentation]    Captures diagnostic screenshots directly into output directory upon failure
    Set Screenshot Directory
    Run Keyword If Test Failed  Capture Page Screenshot    filename=failure-{index}.png
    Delete All Cookies
```

5. Quality Assurance Impact & CI/CD Pipeline

Continuous Integration Workflow (GitHub Actions)

The repository includes a ready-to-run GitHub Actions pipeline (`.github/workflows/regression.yml`) which executes on every push and pull request:

- **Headless Execution:** Provisions Ubuntu runner, installs Google Chrome and dependencies.
- **Dynamic Tag Selection:** Capable of targeting `Smoke` suites for quick sanity checks or full `Regression` runs on demand.
- **Artifact Retention:** Automatically uploads Robot Framework `log.html`, `report.html`, and failure screenshots as build artifacts retained for 14 days.

Measurable Business & Engineering Outcomes

Metric / Area	Before Automation	With Automated Framework	Impact
Regression Run Duration	~3 working days (manual)	~45 minutes (automated)	**~98% faster** feedback loop
Test Data Provisioning	15-20 mins manual form filling	<2 seconds via REST API	Zero repetitive overhead
Attachment Matrix Coverage	Sample testing (2-3 formats)	20+ format automated matrix	100% boundary format coverage
Release Confidence	Frequent post-deploy hotfixes	Zero regression leaks to prod	High stability in citizen portal

Core Competencies Demonstrated

🎯 Test Automation Architecture

Designing modular, layered frameworks using Robot Framework, Python, and Page Object Model patterns.

⚡ Flakiness Mitigation

Implementing dynamic condition-based wait models, viewport scrolling, and clean session teardown.

🔗 Hybrid API + UI Testing

Accelerating test workflows by pre-seeding state via REST endpoints with RequestsLibrary.

🚀 CI/CD & Test Reporting

Configuring headless test execution pipelines in GitHub Actions and publishing HTML reports.

Sanitized Showcase Repository Available

Clean git repository structured and ready for GitHub: `enterprise-portal-automation-robotframework`